Crypto Wall Dashboard — Ruta de implementación

Desde cero absoluto (componentes sin montar) hasta el cuadro colgado en la pared funcionando 24/7.

Fase 0 — Hardware y sistema operativo

El objetivo es tener la Raspberry Pi arrancando, conectada a la pantalla y accesible por SSH desde tu ordenador de trabajo. No toques Python ni código todavía.

0.1 Montar el hardware básico

Conecta la Raspberry Pi 4 a una fuente de alimentación, teclado, ratón y un monitor cualquiera (no la pantalla de 10,1" todavía — usa un monitor normal para configurar cómodamente). Graba Raspberry Pi OS (versión con escritorio) en la microSD usando Raspberry Pi Imager desde tu PC. Arranca, completa el wizard inicial (idioma, WiFi, contraseña del usuario `pi`, actualización del sistema).

0.2 Configurar acceso remoto

Activa SSH desde `raspi-config` o Preferencias del sistema. Activa VNC si quieres ver el escritorio remotamente. Anota la IP local de la Pi (`hostname -I`). A partir de aquí puedes trabajar desde tu ordenador por SSH sin tener monitor/teclado conectados a la Pi, lo cual es mucho más cómodo para programar.

0.3 Conectar la pantalla de 10,1"

Ahora sí, conecta el panel LTN101NT06 a su placa controladora HDMI y esa placa al HDMI de la Pi. Arranca y comprueba que se ve escritorio. Si la resolución no es 1024×600, edita `/boot/config.txt`:

```
hdmi_group=2
hdmi_mode=87
hdmi_cvt=1024 600 60
```

Reinicia y confirma que la resolución es correcta con `xrandr`. No sigas hasta que la pantalla se vea bien.

0.4 Conectar el disco externo USB

Conecta el disco USB que usarás para el nodo DePIN. Formatea como ext4 si no lo está. Configura montaje automático en `/etc/fstab` (por ejemplo en `/mnt/storage`). Reinicia y confirma que el disco se monta solo con `df -h`. Esto no se usa todavía para código, pero es mejor dejarlo listo ahora que estás tocando hardware.

Checkpoint Fase 0: La Pi arranca con la pantalla a 1024×600, el disco USB se monta automáticamente, puedes acceder por SSH desde tu PC.

Fase 1 — Entorno de desarrollo Python

El objetivo es tener el proyecto creado, las dependencias instaladas y un "hello world" en pantalla completa.

1.1 Instalar dependencias del sistema

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv python3-tk unclutter
```

`python3-tk` es necesario para Tkinter. `unclutter` para ocultar el cursor más adelante.

1.2 Crear la estructura del proyecto

En tu PC (no en la Pi), crea el repositorio Git con la estructura de directorios del funcional. Puedes usar GitHub, GitLab o un repo local. La idea es que desarrolles en tu PC y hagas deploy a la Pi con `git pull` o `rsync`.

```
crypto-wall-dashboard/
├── main.py
├── config/
│   ├── __init__.py
│   ├── loader.py
│   └── schema.py
├── services/
│   ├── __init__.py
│   └── base.py
├── ui/
│   ├── __init__.py
│   ├── app.py
│   ├── theme.py
│   └── widgets/
│       ├── __init__.py
│       ├── market_panel.py
│       ├── node_panel.py
│       ├── news_panel.py
│       └── placeholder.py
├── utils/
│   ├── __init__.py
│   ├── formatting.py
│   ├── network.py
│   └── logger.py
├── config.yaml
├── config.example.yaml
├── requirements.txt
└── README.md
```

Todos los archivos pueden estar vacíos o con un `pass` por ahora. Lo importante es que la estructura exista.

1.3 Entorno virtual y dependencias

```
python3 -m venv venv
source venv/bin/activate
pip install requests feedparser matplotlib pyyaml schedule
pip freeze > requirements.txt
```

1.4 Hello World en Tkinter fullscreen

Escribe un `main.py` mínimo que abra una ventana Tkinter fullscreen con fondo oscuro y un texto centrado que diga "Crypto Wall Dashboard". Ejecútalo en la Pi (por VNC o directamente en la pantalla) y confirma que ocupa toda la pantalla a 1024×600 sin bordes ni barra de título.

```
import tkinter as tk

root = tk.Tk()
root.attributes('-fullscreen', True)
root.configure(bg='#0a0a1a')
label = tk.Label(root, text="Crypto Wall Dashboard", fg='#e0e0e0', bg='
label.place(relx=0.5, rely=0.5, anchor='center')
root.mainloop()
```

Si esto se ve bien en la pantalla de 10,1", tienes la base.

Checkpoint Fase 1: Tienes un repo Git con la estructura, dependencias instaladas, y una ventana fullscreen funcionando en la pantalla real.

Fase 2 — Esqueleto de la UI (sin datos reales)

El objetivo es tener los 4 cajones visibles con datos ficticios hardcodeados. Cero llamadas de red.

2.1 Theme y constantes

Implementa `ui/theme.py` con todos los colores, fuentes y tamaños del funcional. Esto se usa en todo el resto de la UI, así que hazlo primero.

2.2 Layout 2×2

En `ui/app.py`, crea la ventana principal y divídela en 4 frames de 512×300 usando `grid` de Tkinter. Cada frame con su color de fondo y un borde sutil. Confirma que los 4 cajones se ven del tamaño correcto.

2.3 Widget de placeholder (Cajón 4)

Empieza por el cajón más simple. Implementa `ui/widgets/placeholder.py` con el texto estático "Próximamente" y las líneas descriptivas. Esto te sirve para establecer el patrón de cómo se construye un widget de cajón: recibe un frame padre, dibuja título y contenido.

2.4 Widget de mercados (Cajón 1) con datos fake

Implementa `ui/widgets/market_panel.py` mostrando:

- Título "Mercados" con el estilo correcto.
- "BTC: $67.234,12" con variación "+2,34%" en verde.
- Un gráfico matplotlib embebido con datos aleatorios (7 puntos, una línea cian sobre fondo oscuro).
- "Oro: $2.051,30 /oz".
- "S&P500: 5.102,45 pts".

Todo hardcodeado. El objetivo es clavar la estética visual.

2.5 Widget de nodo (Cajón 2) con datos fake

Implementa `ui/widgets/node_panel.py` con datos ficticios: badge ONLINE, capacidad, barra de progreso, uptime, tokens. Clava la barra de progreso visual (puede ser un Canvas de Tkinter).

2.6 Widget de noticias (Cajón 3) con datos fake

Implementa `ui/widgets/news_panel.py` con 8-10 noticias hardcodeadas. Implementa la rotación automática (scroll vertical cada 6 segundos). Este es probablemente el widget más complejo de la UI por la animación, así que dale tiempo.

2.7 Indicador de frescura

Añade a cada cajón el punto de estado (verde) y el texto "Actualizado hace X min" en la esquina inferior derecha. Por ahora siempre verde y "hace 0 min".

2.8 Formateo de números

Implementa `utils/formatting.py` con funciones para formatear precios (separador de miles con punto, decimal con coma), porcentajes, y duraciones. Escribe tests manuales rápidos para confirmar que `67234.12` → `67.234,12`.

Checkpoint Fase 2: Los 4 cajones se ven exactamente como el wireframe del funcional, con datos fake, gráfico visible y noticias rotando. La app se ve "terminada" visualmente aunque no tiene datos reales.

Fase 3 — Configuración y logging

El objetivo es que la app lea `config.yaml` al arrancar y tenga logging funcional. Todavía sin llamadas de red.

3.1 Esquema de configuración

Implementa `config/schema.py` con dataclasses (o un dict tipado) que represente la estructura del YAML. Define los valores por defecto.

3.2 Loader con validación

Implementa `config/loader.py` que lee `config.yaml`, lo valida contra el esquema, aplica defaults y devuelve el objeto de configuración. Si el YAML está malformado o no existe, devuelve un error legible (no un traceback).

3.3 Config de ejemplo

Crea `config.example.yaml` completo y documentado con comentarios. El usuario lo copiará a `config.yaml` y rellenará sus API keys.

3.4 Logging

Implementa `utils/logger.py` que configura `RotatingFileHandler` con el formato definido en el funcional. Conecta el logger al `main.py` para que al arrancar se vea en `app.log`:

```
2025-02-10 10:00:00 [INFO] main: Crypto Wall Dashboard arrancando
2025-02-10 10:00:00 [INFO] config: Configuración cargada correctamente
```

3.5 Pantalla de error de configuración

Si `config.yaml` tiene errores, en vez del dashboard, muestra un fondo oscuro con el mensaje de error en texto grande blanco/rojo. El usuario debe poder entender qué va mal sin mirar logs.

3.6 Integrar config con la UI

Modifica la UI para que lea los parámetros del config (colores es fijo, pero la capacidad del nodo, los intervalos, etc. deben venir del YAML). La `capacity_tb` del cajón 2, por ejemplo, debe salir del config.

Checkpoint Fase 3: La app arranca leyendo `config.yaml`, muestra errores legibles si algo falla, y escribe logs. Los datos siguen siendo fake pero los parámetros configurables ya funcionan.

Fase 4 — Servicios: infraestructura base

El objetivo es crear el patrón de servicio que luego replica cada integración.

4.1 Clase base `BaseService`

Implementa `services/base.py` con una clase abstracta que define el ciclo de vida de un servicio:

- `fetch()`: método abstracto que cada servicio implementa (hace la llamada HTTP y parsea).
- `get_cached()`: devuelve el `CachedData` actual.
- `start()`: lanza el loop periódico en un hilo daemon.
- `stop()`: para el loop.
- Retry automático con backoff en caso de fallo.
- Jitter aleatorio al inicio.
- Logging integrado.

4.2 Cliente HTTP compartido

Implementa `utils/network.py` con una función/wrapper de `requests.get` que aplica:

- Timeout configurable (default 10s).
- Headers comunes.
- Logging de cada petición.
- Manejo uniforme de excepciones de red.

4.3 Patrón de actualización UI

Define el mecanismo por el cual un servicio, tras obtener datos nuevos en su hilo, notifica a la UI para que se redibuje. Esto se hace con `root.after_idle()`. Implementa un callback genérico que cada widget registra: "cuando el servicio X tenga datos nuevos, llama a mi método `update_display(data)`".

Checkpoint Fase 4: Tienes una clase base que puedes heredar para cada servicio, un HTTP client limpio, y el mecanismo de actualización hilo→UI probado.

Fase 5 — Servicios: datos de mercado (Cajón 1)

Ahora sí empiezas a hacer llamadas reales. Un servicio cada vez.

5.1 Obtener API keys

Antes de programar, regístrate en los servicios y obtén las API keys:

- CoinGecko: regístrate en el plan Demo (gratis) y obtén tu key.
- Metals.dev (o el proveedor de oro que elijas): regístrate y obtén key.
- Twelve Data (o el proveedor de S&P 500 que elijas): regístrate y obtén key.

Mete las keys en tu `config.yaml`. No hagas commit de este archivo al repo.

5.2 Servicio CoinGecko (BTC)

Implementa `services/coingecko.py` heredando de `BaseService`. Dos llamadas: precio actual y chart 7d. Testea primero desde un script suelto con `requests.get` para confirmar que las URLs, headers y parsing funcionan. Luego intégralo en la clase.

5.3 Conectar BTC al Cajón 1

Sustituye los datos fake de BTC por datos reales del servicio CoinGecko. El precio, la variación 24h y el gráfico ahora se alimentan de la API. Confirma que el gráfico se actualiza cada 5 minutos sin parpadeo ni lag.

5.4 Servicio del Oro

Implementa `services/gold.py`. Mismo patrón: heredar de `BaseService`, fetch, parseo, caché. Conecta al widget.

5.5 Servicio S&P 500

Implementa `services/sp500.py`. Conecta al widget. Comprueba el comportamiento fuera de horario de mercado.

5.6 Manejo de errores en vivo

Ahora prueba desconectando el WiFi/Ethernet de la Pi. Confirma que:

- Los valores se mantienen (caché).
- Los indicadores pasan a amarillo/rojo.
- Al reconectar, los datos se actualizan en el siguiente ciclo.
- No hay crashes ni tracebacks en pantalla.

Checkpoint Fase 5: El Cajón 1 muestra datos reales de BTC, oro y S&P 500, se actualiza periódicamente y tolera fallos de red.

Fase 6 — Servicio RSS (Cajón 3)

6.1 Servicio RSS

Implementa `services/rss.py` heredando de `BaseService`. Usa `feedparser` para parsear los feeds configurados. Implementa la limpieza de títulos (HTML entities, truncado).

6.2 Conectar al Cajón 3

Sustituye las noticias fake por las reales. Confirma que la rotación funciona bien con datos reales (títulos de longitud variable, acentos, caracteres especiales).

6.3 Múltiples feeds

Añade un segundo feed (CoinDesk u otro) y confirma que se unifican y ordenan correctamente.

Checkpoint Fase 6: El Cajón 3 muestra noticias reales de al menos un feed RSS, rotando automáticamente.

Fase 7 — Nodo DePIN (Cajón 2)

7.1 Adaptador Mock

Implementa `services/storage_node.py` con el adaptador `mock` que genera datos ficticios pero que varían ligeramente cada minuto (simula actividad). Esto ya debería funcionar con la UI del cajón 2 que hiciste en la Fase 2.

7.2 Decidir el proyecto DePIN

Ahora es el momento de elegir qué nodo vas a ejecutar (Storj, DeNet, Iagon, etc.). Investiga cuál es compatible con Raspberry Pi 4 + disco USB y cuál tiene mejor documentación/API. Esta decisión no bloquea nada porque el adaptador mock mantiene el cajón funcional.

7.3 Instalar el nodo real (cuando lo elijas)

Instala el software del nodo en la Pi. Configúralo para usar el disco USB montado en `/mnt/`

`storage`. Confirma que el nodo arranca y se conecta a la red.

7.4 Implementar adaptador real

Cuando tengas el nodo corriendo, implementa el adaptador específico (por ejemplo `StorjAdapter`) que lee los datos reales del nodo (vía API local, CLI o logs) y los mapea al `NodeStatus` del funcional.

Checkpoint Fase 7: El Cajón 2 muestra datos del nodo, ya sea mock o reales si ya tienes el nodo instalado.

Fase 8 — Estabilidad y pulido

El objetivo es que la app funcione 72h seguidas sin problemas.

8.1 Test de estabilidad

Deja la app corriendo durante 3 días. Revisa `app.log` cada día buscando errores, warnings o patrones preocupantes. Monitoriza el uso de memoria con `htop` o un script que loguee RSS cada hora — busca memory leaks (crecimiento sostenido de RAM).

8.2 Optimización del gráfico

Si el gráfico de Matplotlib causa picos de CPU o parpadeo al actualizarse, optimiza: reusa el `Figure`, haz downsample de los datos, usa `blit=True` si es posible.

8.3 Pulir animaciones

Ajusta la velocidad de rotación de noticias. Asegúrate de que la transición es suave y no hay saltos. Si Tkinter no puede hacer transiciones suaves, usa un approach más simple (fade o corte limpio).

8.4 Edge cases

Prueba estos escenarios manualmente:

- Arrancar sin conexión a internet.
- Cambiar la `capacity_tb` en el YAML y reiniciar.
- Poner una API key inválida.
- Borrar `config.yaml` y arrancar.
- Dejar correr durante un fin de semana (S&P 500 sin cambios).

8.5 Revisar logs

Limpia logs excesivos de nivel INFO que no aportan. Asegúrate de que los WARNINGs y ERRORs son útiles y accionables.

Checkpoint Fase 8: La app ha corrido 72h sin crashes, el uso de memoria es estable, los logs son limpios.

Fase 9 — Autoarranque y despliegue final

9.1 Configurar autoarranque

Implementa el autostart usando el método que prefieras (LXDE autostart o systemd). Incluye las líneas de `xset` para desactivar screensaver y `unclutter` para ocultar el cursor.

9.2 Test de arranque frío

Apaga la Pi completamente. Desconecta teclado y ratón. Enciéndela solo con pantalla y alimentación. Confirma que el dashboard aparece automáticamente sin ninguna intervención. Hazlo 3 veces.

9.3 Test de recuperación tras reinicio

Fuerza un reinicio (`sudo reboot`). Confirma que la app vuelve sola. Desconecta y reconecta la alimentación (simulando un corte de luz). Confirma que todo vuelve.

Checkpoint Fase 9: La Pi arranca sola y muestra el dashboard sin tocar nada. Se recupera de reinicios y cortes de luz.

Fase 10 — Montaje físico

10.1 Preparar el montaje en pared

Diseña o compra un marco/carcasa para la pantalla de 10,1" con la Pi y la controladora detrás. Puede ser un marco de fotos, una impresión 3D, o una caja de madera/metacrilato. Lo importante es que la pantalla se vea limpia por delante y que haya ventilación por detrás.

10.2 Gestión de cables

Planifica cómo llevar alimentación (USB-C para la Pi, 12V para la controladora de pantalla) y el cable del disco USB. Idealmente todo sale por abajo o por detrás hacia un enchufe cercano.

10.3 Colgar en la pared

Monta el cuadro. Enciende. Disfruta.

10.4 Últimos ajustes visuales

Ahora que lo ves en su ubicación final, ajusta tamaños de fuente si algo no se lee bien a la distancia real. Puede que necesites subir los tamaños un par de puntos.

Checkpoint Fase 10: El cuadro está en la pared, encendido y mostrando datos reales. Proyecto completado.

Resumen visual de la ruta

```
FASE 0  Hardware + OS          ← Raspi funcionando con pantalla
```

```
  |
FASE 1  Entorno Python         ← Repo + deps + hello world fullscreen
  |
FASE 2  UI con datos fake      ← 4 cajones bonitos, cero red
  |
FASE 3  Config + Logging       ← YAML + logs + pantalla de error
  |
FASE 4  Base de servicios      ← Patrón abstracto + threading
  |
FASE 5  APIs de mercado        ← BTC + Oro + S&P reales
  |
FASE 6  RSS                    ← Noticias reales rotando
  |
FASE 7  Nodo DePIN             ← Mock → nodo real (cuando esté)
  |
FASE 8  Estabilidad            ← Test 72h + optimización
  |
FASE 9  Autoarranque           ← Encender = dashboard
  |
FASE 10 Montaje físico         ← Cuadro en la pared 🖼
```

Principio clave de la ruta: En cada fase tienes algo funcional y visible. Nunca pasas más de una fase sin ver resultados en la pantalla. Esto mantiene la motivación y permite detectar problemas pronto.